

2026년 개인주제탐구발표대회 보고서

Ping-Pong Master:
AI는 얼마나 오래 배워야 핑퐁을 잘 칠까?

2026. 9. 4.

과 정 명	AI
소속 학교	대전관평중학교
성 명	김지후

대전교육정보원정보영재교육원

목 차

I. 탐구의 필요성 및 목적

1. 탐구의 필요성
2. 탐구 목적

II. 이론적 배경

1. 선행 연구
2. 용어 정의

III. 탐구방법 및 과정

1. 탐구 방법
2. 탐구 절차
3. 탐구자와 생성형 AI의 역할 분담

IV. 탐구 결과

1. 학습량에 따른 실력 변화
2. 실력이 결정되는 구간과 학습 순서의 영향
3. AI가 배운 규칙
4. 학습 단계별 AI와 대전하는 게임

V. 결론 및 제언

1. 결론
2. 제언

※참고문헌

I. 탐구의 필요성 및 목적

1. 탐구의 필요성

바둑, 게임처럼 사람이 오래 연습해야 잘하게 되는 일을 요즘은 인공지능(AI)이 해낸다. 이런 AI의 상당수는 '강화학습'으로 만들어진다. 강화학습은 정답을 알려 주지 않고 잘하면 보상을 주고 못하면 벌을 주는 것으로 AI가 스스로 방법을 찾아내게 하는 학습 방식이다. 강화학습의 원리를 배우면서 두 가지가 궁금해졌다.

① AI는 얼마나 많이 연습해야 잘하게 될까? 처음에는 못하다가 어느 순간 갑자기 잘해지는 걸까, 아니면 조금씩 좋아지는 걸까?

② 학습을 계속하면 실력도 계속 좋아질까, 아니면 어느 정도에서 멈출까?

이 질문에 답하려면 실력을 숫자로 켈 수 있고 규칙이 단순한 과제가 필요하다. 팽퐁(Pong)은 공을 받아치는 것만 배우면 되고 이겼는지 졌는지가 분명하다. 학습이 끝난 AI와 사람이 직접 대전해 볼 수도 있어 이 탐구에 알맞다고 판단하였다.

2. 탐구 목적

본 탐구의 질문은 “AI를 얼마나 오래 학습시키느냐에 따라 팽퐁 실력은 얼마나 달라질까?”이다. 이를 위해 다음 세 가지를 목표로 하였다.

① 표 기반 Q-learning으로 팽퐁 AI를 직접 학습시킨다. 학습 도중의 AI를 일정 간격으로 저장하여 '학습량이 다른 AI'를 여러 개 확보한다.

② 학습량이 다른 AI들을 같은 상대와 겨루게 하여 승률과 점수 획득률을 켜 뒤 학습량에 따라 실력이 어떻게 변하는지 확인한다.

③ 학습 단계별 AI(초보·중수·고수)와 사람이 직접 대전할 수 있는 브라우저 게임을 만들어 결과를 체험할 수 있게 한다.

II. 이론적 배경

1. 선행 연구

Q-learning은 1989년에 제안된 강화학습 알고리즘이다. 각 상황(상태)에서 각 행동이 얼마나 좋은지를 나타내는 값(Q값)을 표에 적어 두고 경험할 때마다 조금씩 고쳐 나간다. 환경의 규칙을 몰라도 보상만으로 학습할 수 있다는 것이 증명되어 강화학습의 기본 알고리즘으로 자리 잡았다. 상태·행동·보상, 탐험과 활용의 균형 같은 강화학습의 기본 개념을 배우는 출발점으로도 널리 쓰인다.

2015년 DeepMind는 Q값을 표 대신 심층 신경망으로 계산하는 DQN으로 아타리 게임을 화면만 보고 학습시켰다. Pong에서는 사람보다 높은 점수를 얻었다. 그러나 DQN은 수백만 프레임의 학습과 신경망 이해가 필요하다. 본 탐구는 같은 평판을 신경망 없이 1,260칸 표만으로 학습시켜 학습 과정을 표 그대로 들여다본 점이 다르다.

2. 용어 정의

용어	뜻
강화학습	AI가 행동을 해 보고, 그 결과로 받은 보상을 바탕으로 더 좋은 행동을 찾아가는 학습 방식
상태 (state)	지금 게임이 어떤 상황인지를 나타내는 정보. 본 탐구에서는 공의 위치·방향과 네 패들의 위치를 1,260가지로 나눈 것
행동 (action)	AI가 고를 수 있는 선택. 본 탐구에서는 '위로', '가만히', '아래로' 3가지
보상 (reward)	행동의 결과로 받는 점수. 공을 받아치면 +1, 놓치면 -1
Q-표 (Q-table)	행 = 상태, 열 = 행동인 표. 각 칸에는 '이 상태에서 이 행동을 하면 앞으로 얼마나 좋을지'를 나타내는 Q값이 들어 있음
에피소드	서브부터 한쪽이 공을 놓칠 때까지. 학습량은 에피소드 수로 셈
탐험률 (ϵ)	표를 무시하고 아무 행동이나 해 보는 확률. 처음에는 1.0(100%)에서 시작해 0.05까지 줄임
시드(seed)	난수의 시작값. 시드가 다르면 학습 중 겪는 경험 순서가 달라짐
체크포인트	학습 도중 특정 시점의 Q-표를 저장한 것. '학습량이 다른 AI' 하나에 해당

<표 1> 용어 정의

Ⅲ. 탐구방법 및 과정

1. 탐구 방법

가. 게임 환경

핑퐁 게임을 파이썬으로 직접 구현하였다. 화면은 640×400픽셀, 왼쪽 패들이 AI, 오른쪽이 상대(봇 또는 사람)이다. 패들 높이 60px, 공 크기 8px, 패들 속도 240px/s, 공 속도 300px/s이다. 공은 벽에서 반사되며 패들에 맞은 위치에 따라 최대 $\pm 60^\circ$ 로 반사각이 달라진다. 1 에피소드는 서브부터 한쪽이 놓칠 때까지다. 평가와 게임은 11점 먼저 얻는 쪽이 이긴다.

나. 상태·행동·보상

AI가 보는 '상태'는 연속적인 좌표를 그대로 쓰지 않고 <표 2>처럼 네 가지 정보를 칸으로 나누어 만들었다. 조합은 $10 \times 21 \times 2 \times 3 = 1,260$ 가지다. 행동이 3가지이므로 Q-표는 1,260행 × 3열 = 3,780개의 숫자로 이루어진다. 보상은 공을 받아치면 +1, 놓치면 -1(에피소드 종료), 그 외에는 0으로 단순하게

정하였다.

정보	칸 수	의미
공의 x 위치	10	공이 얼마나 가까이 왔나 (64px 단위)
공 y - 내 패들 y	21	공이 내 패들보다 위인지 아래인지, 얼마나 떨어져 있는지 (약 20px 단위)
공의 x 방향	2	나에게 오는 중 / 멀어지는 중
공의 y 방향	3	위로 / 거의 수평 / 아래로

<표 2> 상태를 이루는 네 가지 정보

다. 상대 봇 (AI 훈련 대상)

AI 훈련을 위해서 사람이 계속 게임을 해 주기 어려우므로 상대 봇을 먼저 만들었다. 상대 봇은 공의 y 위치를 따라가는 봇으로, 학습·평가·게임에서 모두 같은 봇을 사용하였다. 봇은 패들 속도의 70%(168px/s)로 움직인다. 사람처럼 불완전하도록 두 가지를 넣었다. 공이 자기 쪽으로 오면서 화면 가운데(x=320)를 넘어야 반응하고 랠리마다 목표 위치에 $\pm 15\text{px}$ 의 무작위 오차가 생기게 하였다.

라. 학습과 평가

Q-learning의 갱신식은 $Q[s,a] \leftarrow Q[s,a] + \alpha(r + \gamma \cdot \max_{a'} Q[s'] - Q[s,a])$ 한 줄이다. 학습 조건은 <표 3>과 같다. 시드 0·1·2로 세 번 반복하였다. 학습 도중 Q-표를 초반 3,000 에피소드까지는 100마다, 그 뒤에는 500마다 저장하여 시드당 64개, 총 192개의 체크포인트를 얻었다.

항목	값	이유
학습률 α	0.1	새 경험을 10%씩 반영
할인율 γ	0.95	수십 프레임 뒤의 결과까지 고려
탐험률 ϵ	1.0 $\rightarrow$ 0.05	15,000 에피소드까지 선형 감소
총 학습량	20,000 에피소드	시드당 약 110초 소요
체크포인트	시드당 64개	학습량이 다른 AI 확보
시드	0, 1, 2	우연과 학습 효과 구분

<표 3> Q-learning 학습 조건

평가는 체크포인트마다 학습한 AI(탐험 없이 표대로만 행동)로 봇과 11점 경기를 30회하여 <표4>의 네 지표 측정하였다. 결과는 3시드 평균 정리하였다.

지표	정의	예시
승률	11점 경기 30회 중 AI가 이긴 경기의 비율. 이겼는지 졌는지만 세트로 실력 차이가 크면 0%나 100%에 붙는다	30경기 중 24승 $\rightarrow$ 80%
점수 획득률	30경기의 전체 득점 중 AI가 얻은 점수의 비율. 한 경기에서 11~21점을 세트로 승률보다 촘촘하게 '얼마나 크게 이겼는지'를 보여 준다	AI 300점 : 봇 200점 $\rightarrow$ 60%

AI 미스율	AI 쪽으로 온 공(AI가 받아쳐야 하는 공) 중 놓친 비율. 승률·점수 획득률과 달리 상대 봇의 실력과 무관하게 AI의 실력만 나타낸다	온 공 100개 중 10개 놓침 → 10%
평균 랠리	1 에피소드(서브부터 한쪽이 놓칠 때까지) 동안 AI가 공을 받아친 횟수의 평균	받아치고 받아치고 놓침 → 2

〈표 4〉 실력을 재는 네 가지 지표

2. 탐구 절차

단계	내용	산출물
1	핑퐁 게임 환경·상태 나누기·봇을 파이썬으로 구현, 자동 테스트 작성	physics.py, env.py, bot.py
2	상대 봇 난이도 조정	봇 미스율 측정
3	Q-learning 학습 3시드 실행	train.py, 체크포인트
4	체크포인트 평가, 학습 곡선, 플레이 영상 제작	eval.csv, 그래프, GIF
5	브라우저 게임 제작	index.html

〈표 5〉 연구 절차

2단계에서 겪은 문제는 탐구 과정에서 가장 중요한 배움이었다. 처음 만든 봇은 공을 거의 완벽하게 따라가서 200번에 1번(0.5%)밖에 공을 놓치지 않았다. 핑퐁은 상대가 놓쳐야 점수가 나므로 이런 봇을 상대로는 AI가 200번에 1번보다 덜 놓치지 않는 한 절대 이길 수 없다. 그래서 학습 중 대부분의 AI는 승률 0%였다. 미스율이 0.5% 아래로 내려간 마지막 순간에만 갑자기 100%가 되는 '계단' 모양이 나왔다. 학습량에 따라 실력이 얼마나 달라지는지를 승률로는 전혀 볼 수 없었다.

승률로 실력 차이를 보려면 상대가 놓치는 확률이 AI가 놓치는 확률과 비슷한 수준이어야 한다는 것을 깨달았다. 예를 들어 상대가 100번에 18번 놓친다면 AI가 100번에 30번 놓칠 때는 지고, 18번이면 반반, 10번이면 이긴다. 실력에 따라 승률이 서서히 변한다. 그래서 봇에 '공이 가까이 와야 반응한다', '목표 위치에 오차가 있다'는 두 가지 약점을 넣었다. 반응 시작 위치 3가지와 오차 크기 3가지를 조합한 9가지 봇을 각각 1,000번씩 시험해 완벽한 AI를 상대로 약 18%를 놓치는 봇을 골랐다.

모든 코드는 파이썬과 numpy만으로 작성하였다. 게임은 같은 물리 계산을 자바스크립트로 옮겨 적었다. 파이썬이 기록한 30초 분량의 공 궤적을 브라우저가 그대로 재현하는지 검사하여 두 구현의 좌표 오차가 0임을 확인하였다.

3. 탐구자와 생성형 AI의 역할 분담

본 탐구는 코드 작성에 생성형 AI(Claude, ChatGPT)를 활용하였다. 탐구의 방향과 규칙은 탐구자가 정하고, 생성형 AI는 그 결정을 코드로 옮기고 결과

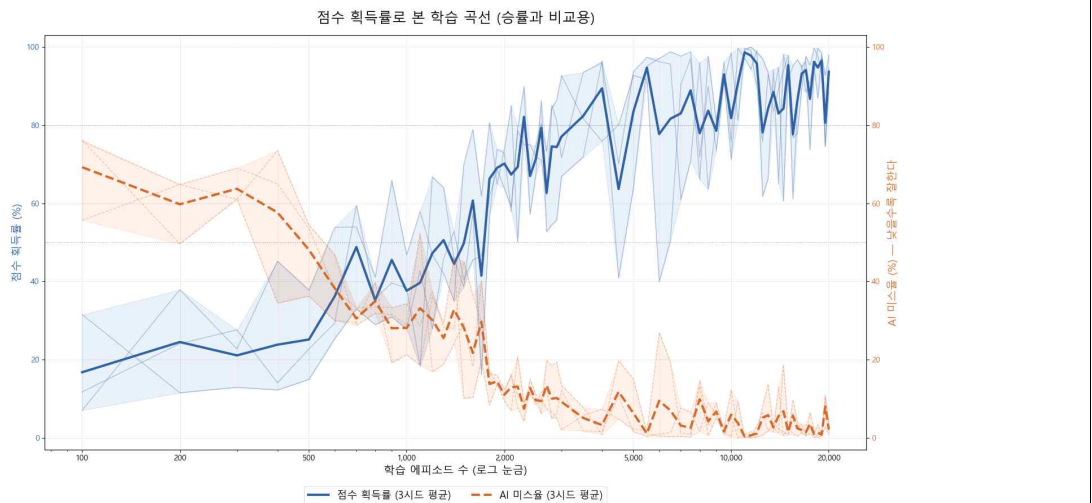
를 정리하는 데 사용하였다. 그 결과는 탐구자가 직접 실행하고 확인하였다(<표 7>). 특히 상대 봇이 지나치게 강해 승률로 실력 차이를 측정하기 어려운 문제를 발견한 뒤, 탐구자가 봇의 반응 시작 위치와 오차 크기를 비교하고 평가 방법을 조정하였다.

구분	수행한 일
탐구자	주제 · 계획, 학습 방식 선택, 상태 · 보상 · 평가 규칙, 실험 설계, 결과 해석, 실행 · 검토
생성형 AI 활용	규칙대로 코딩(환경 · 학습 · 평가 · 게임), 테스트, 시각화, 문서 초안

<표 7> 탐구자와 생성형 AI의 역할 분담

IV. 탐구 결과

1. 학습량에 따른 실력 변화



<그림 1> 학습량에 따른 점수 획득률(파란 실선)과 AI 미스율(주황 점선).

<그림 1>의 가로축은 학습량(에피소드 수), 파란 실선은 점수 획득률(높을수록 AI가 점수를 많이 가져감), 주황 점선은 AI 미스율(낮을수록 공을 잘 받음)이다. 학습 100회일 때 AI는 온 공의 69%를 놓쳐 전체 점수의 17%밖에 얻지 못했다. 2,000회에는 미스율 11%, 점수 획득률 70%로 올라 봇을 이기기 시작했다. 20,000회에는 미스율 2%, 점수 획득률 94%에 이르렀다. 점수 획득률 50%가 봇과 실력이 같은 지점이다. AI는 약 1,500~2,000회 사이에 이 선을 넘었다. <표 6>은 주요 지점의 값이다.

학습량(에피소드)	승률	점수 획득률	AI 미스율	평균 텔리
100	0%	17%	69%	0.4
500	6%	25%	48%	0.8

1,000	19%	38%	28%	1.6
2,000	94%	70%	11%	2.4
5,000	97%	84%	6%	3.0
20,000	100%	94%	2%	3.0

<표 6> 학습량별 실력 (3시드 평균, 11점 경기 30회)

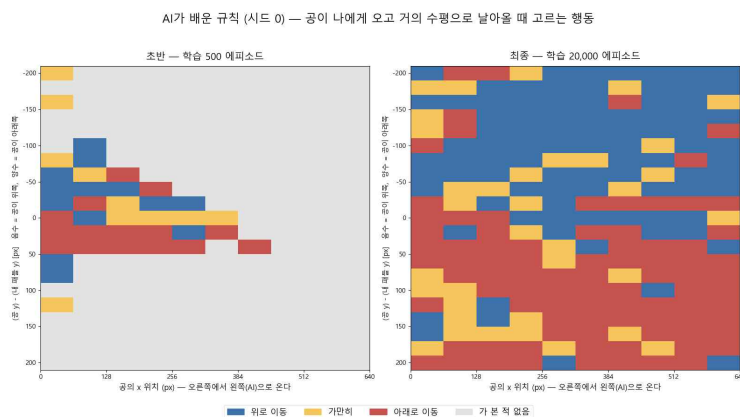
세 시드 모두 학습이 진행될수록 미스율이 내려가고 점수 획득률이 올라갔다. 학습 후반(15,000 이후) 평균 승률은 99.6%였다. 승률은 2,000회 이후 거의 100%에 붙어 그 뒤의 변화를 보여 주지 못한 반면, 점수 획득률은 20,000회까지 계속 올라 '얼마나 크게 이기는지'까지 나타냈다. 평균 랠리는 AI가 거의 놓치지 않게 되면 봇의 실수로만 끝나므로 더 늘지 않는다.

2. 실력이 결정되는 구간과 학습 순서의 영향

승률이 처음 90%에 이른 시점은 시드 0이 1,200회, 시드 1이 900회, 시드 2가 1,900회였다. 즉 전체 학습량 20,000회 중 실력이 갈리는 구간은 앞쪽 10% 남짓이다. 그 뒤는 실력이 천천히 다듬어지는 구간이다.

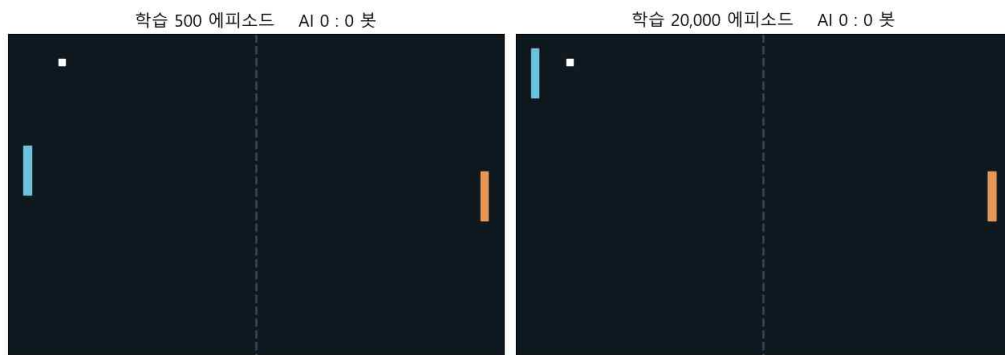
같은 학습량에서도 시드에 따라 실력 차이가 컸다. 1,200회 시점의 승률은 시드 0이 90%, 시드 1이 33%, 시드 2가 0%로 최대 90%p 벌어졌다. 같은 양을 배워도 어떤 순서로 경험했느냐에 따라 실력이 달라진다는 뜻이다. 실험을 한번만 하고 결론을 내리면 위험하다. 또 500~2,000회 구간에서는 승률이 0%와 100% 사이를 오르내리는 진동도 있었다. 이 구간에는 탐험률이 약 0.87~0.97로 아직 높아 임의의 행동을 자주 선택하고 있었으며, Q-표도 충분히 안정되지 않은 시기였기 때문으로 해석하였다.

3. AI가 배운 규칙



<그림 2> 정책 지도(시드 0). 공이 AI 쪽으로 거의 수평으로 올 때, 공의 위치(가로)와 공 · 패들의 높이 차(세로)에 따라 AI가 고르는 행동. 파랑 = 위로, 노랑 = 가만히, 빨강 = 아래로, 회색 = 아직 가 본 적 없는 상태

Q-표는 숫자 3,780개의 표이므로 각 상태에서 Q값이 가장 큰 행동을 색으로 칠하면 AI가 배운 규칙을 한눈에 볼 수 있다. <그림 2>의 왼쪽(학습 500회)은 대부분이 회색이고 색이 있는 칸도 뒤죽박죽이다. 오른쪽(학습 20,000회)에서는 뚜렷한 규칙이 보인다. 세로축은 공의 높이에서 패들의 높이를 뺀 값이므로 위쪽 절반은 '공이 내 패들보다 위에 있는 상황', 아래쪽 절반은 '아래에 있는 상황'이다. 위쪽 절반은 거의 파랑(위로 이동), 아래쪽 절반은 거의 빨강(아래로 이동)으로 나뉜다. '공이 내 위에 있으면 위로, 아래에 있으면 아래로 움직인다'는 규칙을 아무도 가르쳐 주지 않았는데 AI가 보상만으로 스스로 찾아냈다. 노랑(가만히)은 위·아래의 경계 근처와 일부 칸에 흩어져 있을 뿐 뚜렷한 띠를 이루지는 않는다. AI가 공이 정면에 있을 때도 가만히 있기보다 조금씩 움직이며 위치를 맞추기 때문으로 보인다.



<그림 3> 학습 500회 AI(왼쪽)와 20,000회 AI(오른쪽)의 경기 장면. 파란 패들이 AI <그림 3>은 두 AI의 경기 장면을 나란히 기록한 영상의 한 장면이다. 학습 500회 AI는 공과 반대 방향으로 움직이거나 제자리에 머물다 놓치는 일이 잦았다. 20,000회 AI는 공이 오기 전에 미리 자리를 잡아 받아쳤다.

4. 학습 단계별 AI와 대전하는 게임

시드 0의 체크포인트 중 붓 상대 승률이 약 20%, 50%, 80%인 것을 100경기씩 다시 평가하여 초보·중수·고수로 골랐다(<표 8>). 다른 난수로 100경기씩 다시 재어도 고수 > 중수 > 초보의 순서가 유지되었다. 브라우저 게임에서 직접 대전해 보아도 난이도 차이를 체감할 수 있었다.

단계	학습량(에피소드)	승률	점수 획득률	AI 미스율
초보	900	19%	41%	29%
중수	2,800	53%	50%	20%
고수	15,500	83%	60%	10%

<표 8> 브라우저 게임의 난이도 3단계 (시드 0, 11점 경기 100회)

게임은 HTML 파일 하나로 만들어 키보드나 마우스로 AI와 11점 경기를 할 수 있다. 각 레벨의 AI에는 실험에서 측정한 Q-표를 그대로 넣었다.

V. 결론 및 제언

1. 결론

- ① 학습량이 늘수록 실력은 좋아지지만 일정하게 좋아지지는 않는다. 1,000~2,000회 사이에 급격히 올라 붓을 이기기 시작하고, 그 뒤는 미스율이 11%에서 2%로 천천히 다듬어졌다. 실력이 결정되는 시점이 전체 학습량의 10% 근처로 예상보다 훨씬 빨랐다.
- ② 같은 양을 배워도 경험한 순서에 따라 실력이 다르다. 1,200회 시점의 승률이 시드에 따라 0%에서 90%까지 벌어졌으므로 반복 실험과 평균이 필요하다.
- ③ AI는 보상만으로 규칙을 스스로 찾아낸다. '받아치면 +1, 놓치면 -1'만 주었는데 Q-표에 '공이 위면 위로, 아래면 아래로'라는 규칙이 나타났다.
- ④ 실력을 재는 방법 자체가 중요한 부분이다. 붓이 너무 강하면 승률이 계단이 되어 아무것도 보이지 않았다.

2. 제언

- ① 상대 붓이 한 종류뿐이었다. 승률이 약 2,000 에피소드 이후 100% 근처에 도달해 이후의 성능 변화를 보기 어려웠다. 더 강하거나 점점 강해지는 붓을 사용하면 학습 후반의 변화를 더 자세히 볼 수 있을 것이다.
- ② 시드가 3개뿐이어서 학습 초반의 편차를 충분히 다루지 못했다. 학습이 시드당 2분이면 끝나므로 10개 이상으로 늘릴 수 있다.
- ③ 상태를 1,260칸으로 거칠게 나누어 AI가 공의 정확한 위치를 모른다. 칸을 더 촘촘히 나누거나 신경망(DQN)으로 Q값을 계산하면 더 정교한 움직임이 가능한지 비교해 볼 수 있다.

※참고문헌

대전교육정보원 정보영재교육원 AI 수업 자료, 생성형AI(Claude, ChatGPT)

Watkins, C. J. C. H. (1989). Learning from Delayed Rewards. PhD Thesis, University of Cambridge.

Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. Nature, 518, 529 - 533.